

SecureRx

Sistema de Recetas Electrónicas con Criptografía End-to-End

Documentación Técnica Completa

Arquitectura · Redes · Modelo OSI · PKI · Criptografía · API REST · Despliegue

Autor: Kevin Campos

Curso: Criptografía — 7° Semestre

Fecha: Mayo 2026

Índice

1. Introducción y objetivos del proyecto
2. Arquitectura general del sistema
3. Modelo OSI aplicado al proyecto
4. Componentes del stack
 1. Frontend (React + Vite)
 2. nginx (proxy reverso, TLS 1.3)
 3. FastAPI (backend, lógica criptográfica)
 4. MySQL 8.4 (persistencia)
5. Capa de transporte: TLS 1.3
6. Capa de aplicación: criptografía
 1. Argon2id (hashing de contraseñas)
 2. AES-128-GCM (cifrado autenticado)
 3. RSA-OAEP-SHA256 (envoltura de claves)
 4. ECDSA P-256 + SHA3-256 (firmas digitales)
 5. JWT HS256 (autenticación de sesión)
 6. JSON canónico (determinismo)
7. PKI: las dos infraestructuras de clave pública
 1. CA interna del sistema
 2. Certificado TLS auto-firmado
8. Flujos criptográficos completos
 1. Registro y emisión de certificados
 2. Login
 3. Crear receta
 4. Consultar receta (paciente / farmacia)
 5. Dispensar receta
 6. Cancelar y nueva versión
9. Autenticación y RBAC
10. API REST — endpoints
11. Persistencia — modelo de datos
12. Despliegue con Docker Compose
13. Verificación de TLS 1.3
14. Glosario

1. Introducción y objetivos del proyecto

SecureRx es un sistema de recetas electrónicas que aplica criptografía end-to-end para garantizar tres propiedades médico-legales fundamentales:

- **Confidencialidad:** el contenido de la receta solo es legible por el paciente y las farmacias autorizadas. Ni siquiera el servidor puede leerlo.
- **Integridad:** cualquier alteración de la receta (un bit cambiado en dosis, medicamento o cantidad) es detectable.
- **No-repudio:** el médico no puede negar que firmó la receta, y el farmacéutico no puede negar la dispensación. Las firmas son verificables matemáticamente.

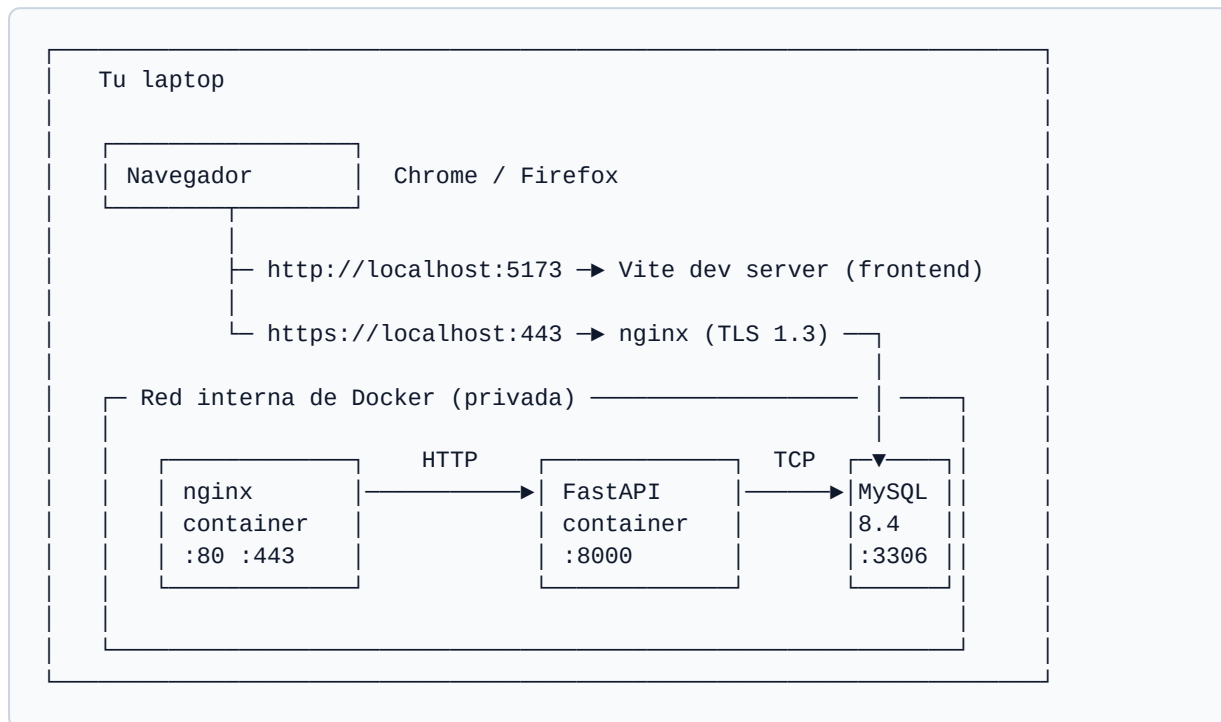
El sistema modela el rol que tendría un sistema de prescripción electrónica conectado a una autoridad sanitaria con CA cualificada (eIDAS en Europa, FIEL en México, etc.). En esta implementación se opera una **CA interna propia** emulando ese rol.

Roles del sistema

Rol	Acciones
admin	Aprueba/rechaza/suspende solicitudes de registro. Opera la CA interna que emite certificados X.509 a usuarios.
medico	Crea recetas firmándolas con ECDSA. Cancela y emite nuevas versiones.
paciente	Recibe y descifra sus recetas. Firma acuses de dispensación.
farmaceutico	Descifra recetas autorizadas, dispensa, firma sello de dispensación.

2. Arquitectura general del sistema

El sistema está compuesto por cuatro componentes que viven en distintos contenedores Docker (con la excepción del frontend de desarrollo, que corre directo en el host con Vite):



El servicio docker-compose se define en `eprescriptions-backend/docker-compose.yml`. Levanta tres contenedores: `db` (MySQL), `api` (FastAPI), y `nginx`. Los volúmenes persistentes son `dbdata` (datos MySQL) y `ca-data` (CA interna).

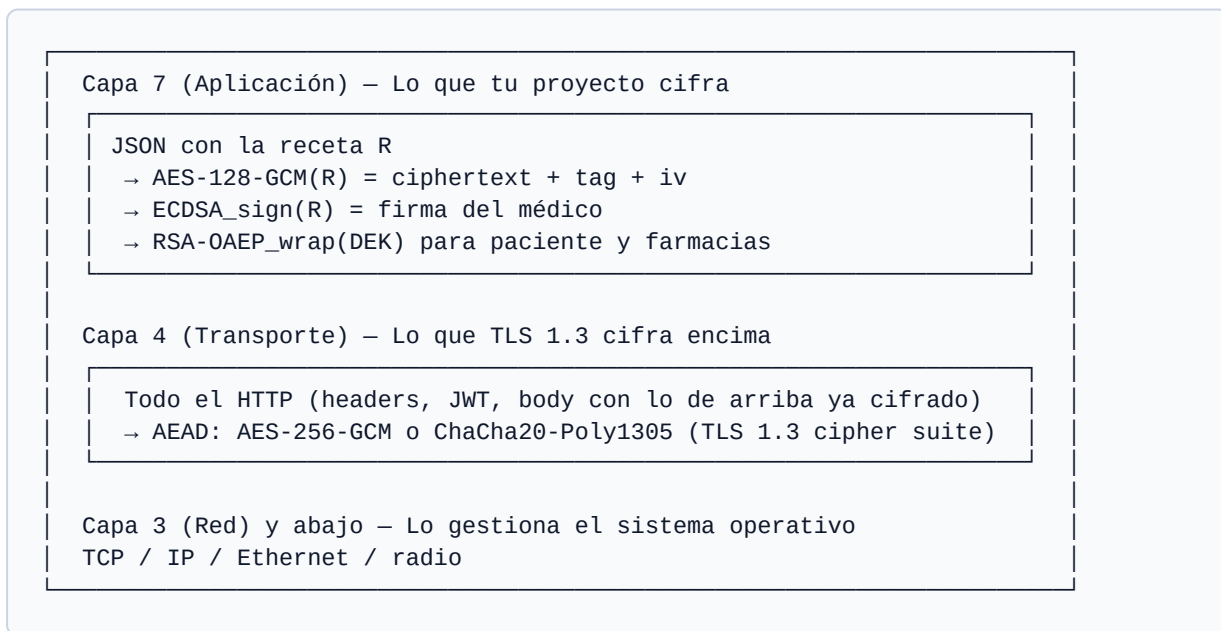
3. Modelo OSI aplicado al proyecto

El modelo OSI organiza la comunicación en redes en 7 capas. No todas son visibles al programador, pero entender dónde "vive" cada parte de SecureRx ayuda a entender por qué hay **dobles cifrados**: uno en transporte (TLS) y otro en aplicación (AES + RSA + ECDSA).

#	Capa	Función	Dónde aparece en SecureRx
7	Aplicación	Protocolos de alto nivel (HTTP, lógica de negocio)	FastAPI, React, criptografía AES-GCM/RSA-OAEP/ECDSA
6	Presentación	Codificación, cifrado, compresión	JSON canónico, Base64, PEM, DER
5	Sesión	Establecimiento y mantenimiento de sesiones	JWT (sesión sin estado), TLS handshake
4	Transporte	Entrega confiable extremo a extremo	TCP, TLS 1.3 (nginx)
3	Red	Direccionamiento (IP, ruteo)	IP del host, red interna de Docker
2	Enlace	Tramas y MAC	Ethernet / Wi-Fi del laptop
1	Física	Bits crudos en el medio	Cable, ondas Wi-Fi

El doble sobre criptográfico

Cuando un médico envía una receta firmada al servidor, el mensaje viaja con DOS capas de protección criptográfica simultáneas:



Por qué hace falta TLS si ya tienes AES + RSA + ECDSA: La criptografía de tu proyecto protege el *contenido* de la receta contra el servidor. TLS protege los *metadatos* (URLs, headers, JWT, llaves privadas en tránsito) contra atacantes en la red. Las dos capas son complementarias — ninguna sustituye a la otra.

4. Componentes del stack

4.1 Frontend (React + Vite)

La interfaz de usuario, escrita en React. Corre en `http://localhost:5173` con el servidor de desarrollo de Vite. Carpeta: `eprescriptions-frontend/`.

- `src/api/index.js` — cliente HTTP (axios) que apunta a `https://localhost` (configurable vía `VITE_API_URL` en `.env`). Inyecta automáticamente `Authorization: Bearer <jwt>` y, cuando aplica, el header `X-Priv-Keys` con el bundle PEM en base64.
- `src/pages/` — pantallas: `Login.jsx`, `Registro.jsx`, `Dashboard.jsx`, `NuevaReceta.jsx`, `MisRecetas.jsx`, `Pendientes.jsx`, `Verificar.jsx` (verifica firmas), `VerificarEmail.jsx`, `RecuperarPassword.jsx`, `AdminSolicitudes.jsx`.
- `src/store/useAuthStore.js` — estado global de autenticación (Zustand) con el JWT y las llaves privadas en memoria.
- `vite.config.js` — proxy de fallback que reenvía `/api/*` a `https://localhost` con `secure: false` (acepta el cert auto-firmado).

4.2 nginx (proxy reverso, TLS 1.3)

nginx es el componente que añade TLS 1.3 al sistema. Termina la conexión cifrada del navegador y reenvía la petición ya descifrada al backend por la red interna de Docker.

Archivo: `eprescriptions-backend/nginx.conf`

```

server {
    listen 443 ssl;
    http2 on;
    server_name localhost;

    ssl_certificate      /etc/nginx/certs/cert.pem;
    ssl_certificate_key  /etc/nginx/certs/key.pem;

    # TLS 1.3 EXCLUSIVAMENTE — sin fallback a 1.2.
    ssl_protocols TLSv1.3;
    ssl_prefer_server_ciphers off;

    add_header Strict-Transport-Security "max-age=63072000" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    location / {
        proxy_pass http://api:8000;
        proxy_set_header Host          $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}

```

Características activas:

- **Solo TLS 1.3** (sin TLS 1.2). Esto fuerza el uso del protocolo más reciente y elimina vectores de ataque conocidos en versiones anteriores.
- **HSTS** con max-age de 2 años — fuerza HTTPS en navegadores que ya conectaron una vez.
- **X-Content-Type-Options: nosniff** — previene MIME-sniffing.
- **X-Frame-Options: DENY** — previene clickjacking.
- **HTTP/2** activo (multiplexing de múltiples requests sobre una sola conexión TLS).

4.3 FastAPI (backend, lógica criptográfica)

El núcleo del sistema. Corre Python 3.12 con FastAPI. Toda la lógica de negocio y criptografía de aplicación vive aquí.

Punto de entrada: `eprescriptions-backend/main.py`


```

app = FastAPI(title="SecureRx – Sistema de Recetas Electrónicas", version="2.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=_origins,          # whitelist desde CORS_ORIGINS
    allow_credentials=True,
    allow_methods=["GET", "POST", "PATCH", "DELETE", "OPTIONS"],
)

app.include_router(health.router)
app.include_router(usuarios.router)      # §1, §3
app.include_router(admin.router)         # §2
app.include_router(recetas_crear.router)  # §4
app.include_router(recetas_consulta.router) # §5
app.include_router(recetas_dispensar.router) # §6
app.include_router(recetas_cancelar.router) # §8
app.include_router(recetas_nueva_version.router) # §9

```

Estructura de directorios del backend:

```

eprescriptions-backend/
├── main.py                ← arranque + montaje de routers
├── auth.py                ← auth_required, require_roles (RBAC)
├── audit.py               ← log transaccional de eventos
├── database.py            ← modelos SQLAlchemy (Usuario, Receta, ...)
├── routers/
│   ├── usuarios.py        ← §1 registro, §3 login, recuperar pw
│   ├── admin.py           ← §2 emitir/revocar certs
│   ├── recetas_crear.py    ← §4 crear receta
│   ├── recetas_consulta.py ← §5 consultar / verificar firmas
│   ├── recetas_dispensar.py ← §6 dispensar + acuse paciente
│   ├── recetas_cancelar.py ← §8 cancelar
│   └── recetas_nueva_version.py ← §9 sustituir receta
├── services/
│   ├── crypto/
│   │   ├── aes_gcm.py
│   │   ├── rsa_oaep.py
│   │   ├── signatures.py  ← ECDSA
│   │   ├── argon2_pw.py
│   │   ├── jwt_service.py
│   │   ├── canonical.py   ← JSON canónico
│   │   ├── ca.py          ← CA interna X.509
│   │   └── keys.py        ← generación/serialización PEM
│   ├── bundle.py          ← parsing del header X-Priv-Keys
│   ├── canonical_receta.py ← R canónico, AAD, sello, M_cancel
│   ├── receta_cifrado.py  ← AES + wrap RSA en una operación
│   └── receta_descifrado.py ← inverso del anterior
└── schemas/               ← contratos Pydantic

```

4.4 MySQL 8.4 (persistencia)

Solo guarda lo que NO compromete la confidencialidad: hashes de passwords, llaves públicas, certificados X.509, ciphertext de recetas, firmas, audit log. Las llaves privadas y los contenidos en claro de las recetas *nunca* tocan la base de datos.

Modelo: `eprescriptions-backend/database.py` . Tablas principales:

Tabla	Contenido
<code>usuarios</code>	<code>username</code> , <code>email</code> , <code>password_hash</code> (Argon2id), <code>salt_pw</code> , <code>rol</code> , <code>estado</code> , <code>pub_ec_pem</code> , <code>pub_rsa_pem</code>
<code>solicitudes_certificado</code>	solicitudes pendientes/aprobadas/rechazadas con snapshot histórico
<code>certificados</code>	X.509 v3 emitidos, serial, expiración, flag de revocación
<code>recetas</code>	<code>ciphertext</code> , <code>tag_aes</code> , <code>iv_aes</code> , <code>aad</code> , <code>c_wrap_pac</code> , <code>firma_doctor</code> , <code>hash_sha3_hex</code>
<code>receta_acceso_farmacias</code>	<code>c_wrap_far</code> : la DEK envuelta para cada farmacia autorizada
<code>eventos_dispensacion</code>	<code>firma_sello</code> (farmacéutico), <code>firma_paciente</code> , manifiesto JSON canónico
<code>cancelaciones</code>	<code>manifiesto M_cancel</code> + <code>firma_cancel</code>
<code>audit_logs</code>	traza inmutable de todas las acciones

5. Capa de transporte: TLS 1.3

TLS (Transport Layer Security) es el protocolo que añade confidencialidad, integridad y autenticación de servidor a cualquier conexión TCP. Vive entre las capas 4 (transporte) y 7 (aplicación) del modelo OSI. La versión 1.3 (RFC 8446, 2018) es la actual.

5.1 Diferencias clave de TLS 1.3 vs 1.2

Aspecto	TLS 1.2	TLS 1.3
Round-trips para handshake	2	1 (hasta 0 con 0-RTT)
Cipher suites	cientos, muchas inseguras	5 modernas (todas AEAD)
Algoritmos eliminados	—	RC4, 3DES, MD5, SHA-1, RSA key exchange
Forward secrecy	opcional	obligatorio (solo (EC)DHE)
Compresión	vulnerable (CRIME)	eliminada

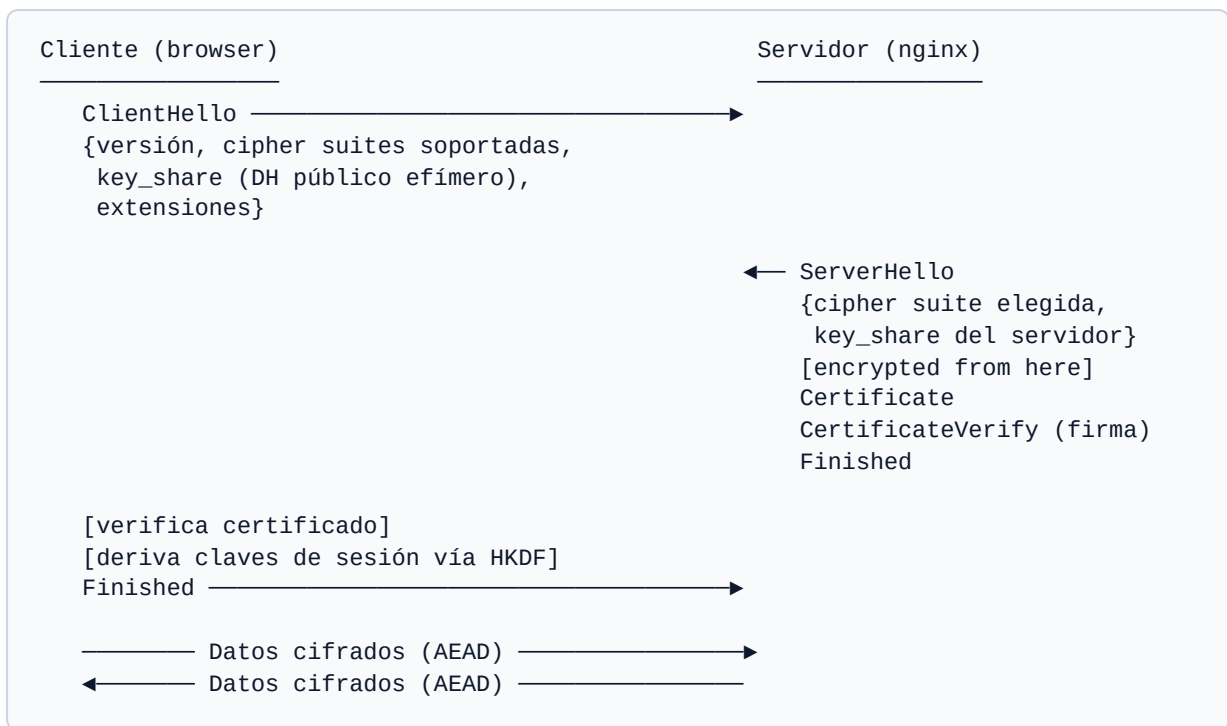
5.2 Cipher suites de TLS 1.3

TLS 1.3 deja solo 5 cipher suites, todas con AEAD:

- TLS_AES_256_GCM_SHA384
- TLS_CHACHA20_POLY1305_SHA256
- TLS_AES_128_GCM_SHA256
- TLS_AES_128_CCM_8_SHA256
- TLS_AES_128_CCM_SHA256

nginx negocia con el navegador y eligen la primera que ambos soportan. Lo más común con Chrome/Firefox modernos: TLS_AES_256_GCM_SHA384 .

5.3 El handshake TLS 1.3 (1-RTT)



A partir del segundo flight, todo el contenido va cifrado con la cipher suite negociada. La **forward secrecy** está garantizada porque las llaves DH son efímeras — si en el futuro alguien roba la llave privada del servidor, no puede descifrar conexiones pasadas porque las llaves DH ya fueron destruidas.

5.4 Configuración del cert TLS en SecureRx

El certificado del servidor lo genera el script `eprescriptions-backend/gen-cert.sh`:

```
openssl ecparam -genkey -name prime256v1 -out certs/key.pem

openssl req -new -x509 \
  -key certs/key.pem \
  -out certs/cert.pem \
  -days 365 \
  -subj "/CN=localhost/O=SecureRx/C=MX" \
  -addext "subjectAltName=DNS:localhost,DNS:127.0.0.1,IP:127.0.0.1"
```

Es un cert **auto-firmado ECDSA P-256** con vigencia de 365 días. El navegador muestra warning la primera vez (porque no proviene de una CA pública), pero el TLS 1.3 que se establece es *real* y *criptográficamente válido*.

Auto-firmado ≠ inseguro: el cert es auto-firmado solo en el sentido de que lo firma su propia llave privada en lugar de una CA pública. La criptografía del handshake, las claves DH efímeras, el cifrado AES-GCM o ChaCha20-Poly1305: todo igual de seguro que en un sitio "real". Lo único que cambia es que el navegador no confía automáticamente.

5.5 Verificación de TLS 1.3

Para confirmar que la conexión usa TLS 1.3:

```
curl -v --tlsv1.3 -k https://localhost/health
```

En la salida aparecerán líneas como:

```
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN: server accepted h2
```

Eso es prueba criptográfica de que el handshake TLS 1.3 se completó.

6. Capa de aplicación: criptografía

Esta es la sección donde vive el corazón del proyecto: la criptografía que protege a la receta contra el propio servidor. Todas las primitivas viven en `eprescriptions-backend/services/crypto/`.

6.1 Argon2id (hashing de contraseñas)

Archivo: `services/crypto/argon2_pw.py`

Argon2id es el ganador del Password Hashing Competition (2015) y el estándar moderno para hashear contraseñas. Combina los modos Argon2i (resistente a side-channel) y Argon2d (resistente a GPU/ASIC).

```
_HASHER = PasswordHasher(
    time_cost=3,          # 3 iteraciones
    memory_cost=64 * 1024, # 64 MiB de RAM por verificación
    parallelism=4,         # 4 threads paralelos
    hash_len=32,           # 32 bytes de salida
    salt_len=32,           # 32 bytes de salt aleatorio
)
```

Cada vez que un usuario se registra, el password se hashea con esos parámetros. El salt aleatorio se incluye dentro del string Argon2 final (formato `$argon2id$v=19$m=...,t=3,p=4$<salt>$<hash>`). Verificar el password requiere ~64 MiB de RAM por intento, lo que hace inviable el ataque de fuerza bruta con GPUs.

6.2 AES-128-GCM (cifrado autenticado)

Archivo: `services/crypto/aes_gcm.py`

AES-GCM (Galois/Counter Mode) es un cifrado autenticado con datos asociados (AEAD). Combina el cifrado AES en modo CTR con un MAC basado en multiplicación en $GF(2^{128})$. Output: ciphertext + tag de autenticación de 128 bits.

```
def aes_gcm_encrypt(key: bytes, iv: bytes, plaintext: bytes, aad: bytes):
    if len(key) != 16: raise ValueError("DEK debe ser de 128 bits (16 bytes)")
    if len(iv) != 12:  raise ValueError("IV debe ser de 96 bits (12 bytes)")
    ct_tag = AESGCM(key).encrypt(iv, plaintext, aad)
    return ct_tag[:-16], ct_tag[-16:] # (ciphertext, tag)
```

Parámetros:

- **Llave (DEK):** 128 bits, generada con `secrets.token_bytes(16)` (CSPRNG del SO).
- **IV:** 96 bits aleatorio (12 bytes), único por mensaje.
- **AAD:** datos adicionales autenticados (no cifrados pero protegidos por el TAG).
- **TAG:** 128 bits, lo verifica el destinatario al descifrar.

Si un solo bit del ciphertext, el IV o el AAD se altera, la verificación del TAG falla y el descifrado lanza excepción. Esto es lo que hace que AES-GCM sea AEAD.

6.3 RSA-OAEP-SHA256 (envoltura de claves)

Archivo: `services/crypto/rsa_oaep.py`

RSA-OAEP (Optimal Asymmetric Encryption Padding) es el padding seguro estándar para cifrado RSA. Lo usamos para "envolver" la DEK del paciente o de cada farmacia, de forma que solo el dueño de la llave privada RSA correspondiente pueda recuperar la DEK.

```
_PADDING = padding.OAEP(
    mgf=padding.MGF1(algorithm=hashes.SHA256()),
    algorithm=hashes.SHA256(),
    label=None,
)

def rsa_oaep_encrypt(pub_pem: str, dek: bytes) -> bytes:
    pub = serialization.load_pem_public_key(pub_pem.encode())
    return pub.encrypt(dek, _PADDING)
```

Tamaño de llave: **RSA 2048 bits**, con exponente público 65537 (estándar).

¿Por qué RSA-OAEP además de AES?

AES es simétrico: misma llave para cifrar y descifrar. Si solo hubiera AES, el médico tendría que compartir la DEK con paciente y farmacias por algún canal seguro pre-establecido. RSA-OAEP resuelve esto: **cifras la misma DEK con la pública de cada destinatario**. Cada uno usa su propia privada para recuperarla. Patrón clásico de cifrado híbrido.

6.4 ECDSA P-256 + SHA3-256 (firmas digitales)

Archivo: `services/crypto/signatures.py`

```
def ecdsa_sign(priv: EllipticCurvePrivateKey | str, message: bytes) -> str:
    if isinstance(priv, str):
        priv = serialization.load_pem_private_key(priv.encode(), password=None)
    sig_der = priv.sign(message, ec.ECDSA(hashes.SHA3_256()))
    return base64.b64encode(sig_der).decode()
```

ECDSA = Elliptic Curve Digital Signature Algorithm. Curva: **SECP256r1** (alias NIST P-256). Hash: **SHA3-256** (Keccak, FIPS 202). Firma serializada como DER y luego Base64 para transporte en JSON.

Operaciones:

- `ecdsa_sign(priv, mensaje)` → produce firma única, no determinística (ECDSA aleatorio).
- `ecdsa_verify(pub_pem, mensaje, firma_b64)` → bool, atómica.

Cualquiera con la llave pública del firmante puede verificar. Solo el dueño de la privada puede generar firmas válidas. Esa es la base del **no-repudio**: el médico no puede negar haber firmado una receta porque solo él tiene la llave que produce esa firma.

6.5 JWT HS256 (autenticación de sesión)

Archivo: `services/crypto/jwt_service.py`

```
def sign_jwt(id_usuario: int, rol: str) -> str:
    payload = {
        "id_usuario": id_usuario,
        "rol": rol,
        "exp": int(time.time()) + _EXP_MIN * 60,
        "iat": int(time.time()),
    }
    return jwt.encode(payload, _require_secret(), algorithm="HS256")
```

Tras un login exitoso, el backend emite un JWT firmado con HMAC-SHA256 usando un secreto del servidor (variable `JWT_SECRET`, mínimo 32 caracteres). El cliente lo manda en cada petición como `Authorization: Bearer <jwt>`. El backend lo verifica en `auth.py` y carga el usuario.

6.6 JSON canónico

Archivo: `services/crypto/canonical.py`

```
def canonical_json(obj):
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=
```


Un detalle sutil pero crítico: para que dos partes puedan firmar/verificar el mismo dict y obtener la misma firma, deben serializarlo **byte-a-byte idéntico**. Diferencias de espacios, orden de claves o codificación romperían la verificación.

Por eso todo lo que se firma o se usa como AAD pasa por `canonical_json`: claves ordenadas alfabéticamente, sin espacios extra, UTF-8. Funciones derivadas en `services/canonical_receta.py`: `build_R` (receta canónica), `build_AAD`, `build_sello`, `build_M_cancel`.

7. PKI: las dos infraestructuras de clave pública

SecureRx maneja DOS infraestructuras de clave pública independientes que es importante no confundir.

7.1 PKI #1 — CA interna del sistema (firma certs de usuarios)

Archivo: `services/crypto/ca.py`

Cuando el admin aprueba una solicitud de registro, la CA interna emite **dos certificados X.509 v3** al usuario:

- Cert **EC** con `KeyUsage = digitalSignature, contentCommitment` (firma).
- Cert **RSA** con `KeyUsage = keyEncipherment` (cifrado/wrap).

```
def emitir_cert(pub_pem, nombre, rol, uso):
    ca_key, ca_cert = _ensure_ca()          # raíz EC P-256, on-first-boot
    pub = serialization.load_pem_public_key(pub_pem.encode())
    subject = x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, nombre),
        x509.NameAttribute(NameOID.ORGANIZATIONAL_UNIT_NAME, rol),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SecureRx"),
    ])
    builder = (x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(ca_cert.subject)
        .public_key(pub)
        .serial_number(x509.random_serial_number())
        .not_valid_before(now - timedelta(minutes=5))
        .not_valid_after(now + timedelta(days=365))
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True,
            # ... KeyUsage según uso = "firma" | "cifrado"
        )
    )
    cert = builder.sign(private_key=ca_key, algorithm=hashes.SHA256())
```

La raíz de la CA (su llave privada y su cert) vive en el volumen persistente `ca-data`, montado en `/app/ca`. Se genera automáticamente al primer arranque y se reusa después. La **llave privada de la CA jamás sale del servidor**.

7.2 PKI #2 — Certificado TLS del servidor (auto-firmado)

Archivo: `eprescriptions-backend/certs/cert.pem` + `key.pem`

Independiente de la CA del sistema. Es el cert que nginx presenta en el handshake TLS 1.3. Generado por `gen-cert.sh`, también ECDSA P-256 con SAN para `localhost` + `127.0.0.1`.

¿Por qué auto-firmado y no Let's Encrypt?

Las CAs públicas (Let's Encrypt, DigiCert, etc.) **no firman certificados para `localhost`** porque no pueden verificar que tú eres dueño de algo que no existe en internet. Para usar Let's Encrypt necesitarías:

1. Comprar un dominio público (~\$10/año).
2. Apuntar su DNS a un servidor con IP pública.
3. Correr `certbot` o usar Caddy/Traefik con auto-renovación.

En el contexto de un proyecto académico que corre local, el cert auto-firmado es **la elección correcta**, no una limitación. Si el sistema se desplegara con dominio público, solo cambiaría esa pieza — el resto de la criptografía permanecería idéntica.

	PKI #1 — CA interna	PKI #2 — TLS
Qué firma	Llaves públicas de usuarios (médicos, pacientes, farmacias)	Llave pública de nginx
Quién la opera	Admin de SecureRx	nginx/sysadmin
Validez	365 días, dentro del sistema	365 días, presentado en handshake TLS
Reemplazable por CA pública	No (las CAs públicas no certifican personas con rol)	Sí, en producción con dominio (Let's Encrypt)

8. Flujos criptográficos completos

8.1 Registro y emisión de certificados

1. Usuario manda `POST /usuarios/registro` con username, nombre, email, password, rol.
2. El backend (`routers/usuarios.py`):
 - Valida unicidad de username/email.
 - Genera par EC P-256 y par RSA 2048.
 - Calcula `password_hash = Argon2id(password, salt_pw)`.
 - Inserta el usuario con estado `pendiente`.
 - Crea una `SolicitudCertificado` ligada.
 - Envía email de verificación con un token URL-safe de 32 bytes.
 - Devuelve al cliente el bundle PEM con **ambas llaves privadas** (UNA sola vez).
3. El admin revisa solicitudes en `GET /admin/solicitudes` y aprueba con `POST /admin/solicitudes/{id}/aprobar`. La CA emite los dos certs y el usuario pasa a estado `activo`.

8.2 Login

Archivo: `routers/usuarios.py` función `login`.

1. El usuario manda username, password, rol, llave_privada_ec, llave_privada_rsa.
2. Backend:
 - `verify_password(hash, password)` con Argon2 — tiempo constante.
 - Verifica que el rol declarado coincida con el de BD.
 - Verifica que tenga certs EC y RSA vigentes y no revocados.
 - Para roles $\neq$ admin: deriva las llaves públicas de las privadas enviadas y compara contra las registradas en BD (`bundle.exigir_ec`, `bundle.exigir_rsa`).
 - Emite JWT con `{id_usuario, rol, exp, iat}` y lo devuelve.

8.3 Crear receta (§4)

Archivo: `routers/recetas_crear.py`



8.4 Consultar receta (paciente)

Archivo: `routers/recetas_consulta.py`

1. Cliente manda `GET /recetas/paciente/{id}` con header `X-Priv-Keys` (bundle PEM en base64).
2. Backend:
 - Valida JWT, rol=paciente, id coincide.
 - Verifica pertenencia de RSA priv.
 - Para cada receta del paciente: `DEK = RSA_OAEP_unwrap(rsa_priv, c_wrap_pac)`, luego `R = AES_GCM_decrypt(DEK, IV, ct, TAG, AAD)`.
 - Devuelve el JSON descifrado.

8.5 Dispensar receta (§6)

Archivo: `routers/recetas_dispensar.py`

Farmacéutico

Backend

POST /recetas/{id}/dispensar

- Validar JWT, rol=farmaceutico
- Verificar pertenencia EC y RSA del farmacéutico
- Comprobar autorización (RecetaAccesoFarmacia)
- DEK = RSA_OAEP_unwrap(rsa_priv_farm, c_wrap_far)
- R = AES_GCM_decrypt(DEK, IV, ct, TAG, AAD)
- verify_ECDSA(pub_ec_medico, R, firma_doctor) ← anti-forgery
- sello = canonical_json({
 id_farmaceutico, id_receta, numero_dispensacion,
 timestamp_iso, dispensaciones_permitidas
})
- firma_sello = ECDSA_sign(priv_ec_farm, sello)
- INSERT eventos_dispensacion (firma_sello, manifiesto_sello)
 UPDATE recetas SET dispensaciones_realizadas++,
 estado = 'en_proceso' o 'dispensada_completa'

El paciente luego firma el mismo manifiesto del sello como acuse de recibo (POST /recetas/eventos-dispensacion/{id}/firmar-paciente). Esto produce evidencia no-repudiable de tres partes: médico (sobre R), farmacéutico (sobre el sello), paciente (sobre el sello).

8.6 Cancelar y nueva versión (§8, §9)

Cancelar (`routers/recetas_cancelar.py`): el médico construye `M_cancel = canonical_json({id_receta, id_doctor, motivo, timestamp, dispensaciones_realizadas_al_cancelar})` y lo firma con ECDSA. Se almacena en la tabla `cancelaciones`.

Nueva versión (`routers/recetas_nueva_version.py`): cancela la original (idéntico a §8) y emite una nueva receta con `parent_id` apuntando a la sustituida. Material criptográfico totalmente fresco (DEK e IV nuevos, firma nueva).

9. Autenticación y RBAC

Archivo: `eprescriptions-backend/auth.py`

```
def auth_required(authorization: str | None = Header(default=None),
                  db: Session = Depends(get_db)) -> Usuario:
    token = _extract_token(authorization)
    payload = verify_jwt(token)
    uid = payload.get("id_usuario")
    rol_tok = payload.get("rol")
    u = db.query(Usuario).filter(Usuario.id == uid).first()
    if not u or not u.activo or u.estado != "activo":
        raise HTTPException(403, "Cuenta inactiva")
    if u.rol != rol_tok:
        raise HTTPException(403, "Rol no coincide") # defensa: rol cambió en BD
    return u

def require_roles(*roles: str):
    def _dep(user: Usuario = Depends(auth_required)) -> Usuario:
        if roles and user.rol not in roles:
            raise HTTPException(403, f"Rol requerido: {'/'.join(roles)}")
        return user
    return _dep
```

Cada endpoint sensible declara qué rol(es) acepta: `require_roles("medico")`, `require_roles("farmaceutico")`, etc. Defensa adicional: si el rol del JWT no coincide con el rol actual en BD (porque el admin lo cambió), se rechaza la petición.

Anti-IDOR: endpoints como `POST /recetas?medico_id=X` verifican que el query param coincida con el `user.id` del JWT, evitando que el médico A actúe en nombre del médico B.

10. API REST — endpoints principales

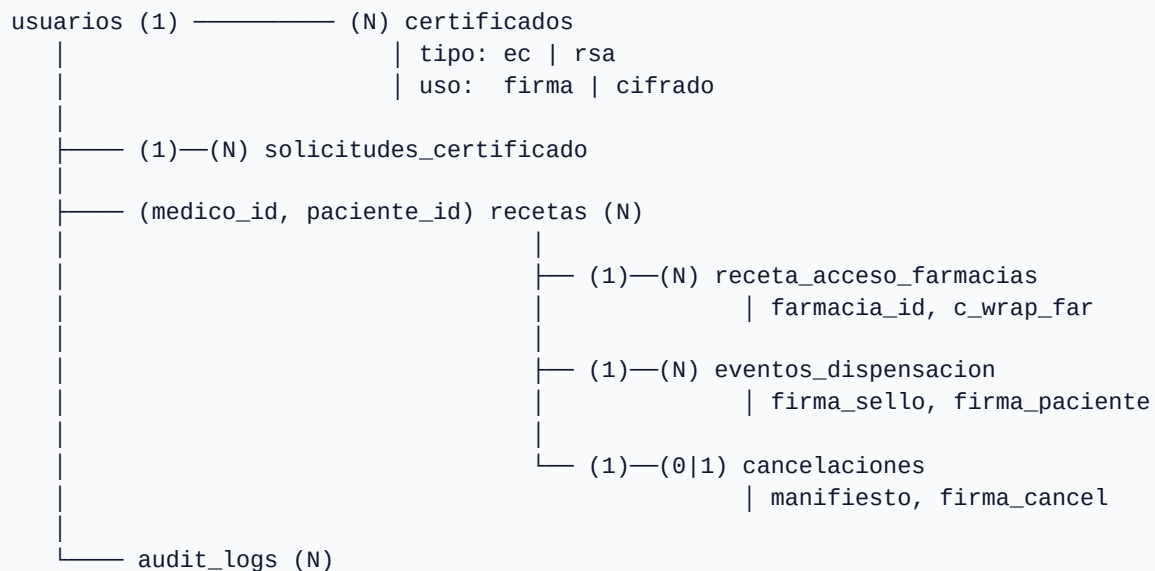
Método	Ruta	Rol	Descripción
POST	/usuarios/registro	público	Registro con generación de par EC + RSA
POST	/usuarios/login	público	Login con verificación Argon2 + pertenencia de llaves
POST	/usuarios/recuperar-password	público	Inicia flujo de reseteo (email)
POST	/usuarios/reset-password	público	Aplica nueva password con token
GET	/usuarios/verificar-email	público	Verifica email con token
GET	/admin/solicitudes	admin	Listar solicitudes pendientes/etc.
POST	/admin/solicitudes/{id}/aprobar	admin	Emite los 2 certs X.509 y activa al usuario
POST	/admin/solicitudes/{id}/suspender	admin	Suspende usuario y revoca certs
POST	/admin/solicitudes/{id}/rechazar	admin	Borra usuario sin historial
POST	/recetas?medico_id=X	medico	Crear receta cifrada y firmada
GET	/recetas/paciente/{id}	paciente	Listar y descifrar recetas propias
GET	/recetas/farmaceutico/{id}	farmaceutico	Histórico de dispensaciones
GET	/recetas/pendientes	farmaceutico	Recetas activas para dispensar
POST	/recetas/{id}/dispensar?farmaceutico_id=X	farmaceutico	Dispensar receta + firma sello
POST	/recetas/eventos-dispensacion/{id}/firmar-paciente	paciente	Acuse del paciente sobre el sello

POST	/recetas/{id}/cancelar	medico	Cancela receta con M_cancel firmado
POST	/recetas/{id}/nueva-version	medico	Sustituye receta (cancela + crea nueva)
GET	/recetas/{id}/verificar-firmas	auth	Verifica firmas públicas
GET	/ca/certificate	público	Cert público de la CA interna
GET	/health	público	Healthcheck

11. Persistencia — modelo de datos

Archivo: `eprescriptions-backend/database.py`

SQLAlchemy 2.x sobre MySQL 8.4. Relaciones principales:



Lo que **NO** está en BD (principio de "lo que nunca llega al servidor"):

- Llaves privadas EC ni RSA de usuarios.
- Contenido en claro de las recetas.
- Passwords en claro (solo hash Argon2).
- DEKs (se generan, usan, y se borran de memoria).

12. Despliegue con Docker Compose

Archivo: `eprescriptions-backend/docker-compose.yml`

Tres servicios y dos volúmenes persistentes:

Servicio	Imagen	Puertos	Volúmenes
db	mysql:8.4	3306 (interno)	dbdata:/var/lib/mysql
api	build local (python:3.12-slim)	8000 (interno)	ca-data:/app/ca
nginx	nginx:1.27-alpine	8080 → 80, 443 → 443	./certs, ./nginx.conf (read-only)

Script de arranque `start.sh`:

1. Verifica dependencias (docker, npm, openssl).
2. Genera el cert TLS si no existe.
3. Crea `.env` con `JWT_SECRET` aleatorio si falta.
4. Levanta `docker compose up -d --build`.
5. Espera healthcheck del api.
6. Prueba el handshake TLS 1.3 contra `https://localhost/health`.
7. Instala `node_modules` del frontend si falta.
8. Arranca `npm run dev` en foreground.
9. Trap `Ctrl+C` apaga frontend + backend de forma limpia.

13. Verificación de TLS 1.3

Comandos para confirmar que la conexión real usa TLS 1.3:

13.1 con curl

```
curl -v --tlsv1.3 --tls-max 1.3 -k https://localhost/health
```

Salida esperada (líneas relevantes):

```
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN: server accepted h2
< HTTP/2 200
{"status":"ok"}
```

13.2 con openssl s_client

```
openssl s_client -connect localhost:443 -tls1_3
```

Salida esperada incluye:

```
SSL handshake has read X bytes and written Y bytes
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 256 bit
Protocol : TLSv1.3
```

13.3 captura de paquetes con Wireshark

Filtro `tcp.port == 443`. En el primer paquete de respuesta del servidor verás:

- **ServerHello** con `Version: TLS 1.2 (legacy)` pero `supported_versions extension: TLS 1.3`.
- Después del handshake, los paquetes son `Application Data` con contenido cifrado.

14. Glosario

Término	Definición
AAD	Additional Authenticated Data — datos no cifrados pero autenticados por el TAG en AEAD.
AEAD	Authenticated Encryption with Associated Data — cifrado autenticado (AES-GCM, ChaCha20-Poly1305).
CA	Certificate Authority — emite certificados X.509 firmando llaves públicas.
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator (Python: <code>secrets</code>).
DEK	Data Encryption Key — la llave AES que cifra el contenido real.
DH / ECDHE	Diffie-Hellman / EC ephemeral — acuerdo de claves con forward secrecy.
ECDSA	Elliptic Curve Digital Signature Algorithm.
HKDF	HMAC-based Key Derivation Function (RFC 5869) — usado dentro de TLS 1.3.
HSTS	HTTP Strict Transport Security — fuerza HTTPS en navegadores.
JWT	JSON Web Token — token de sesión firmado.
OAEP	Optimal Asymmetric Encryption Padding — padding seguro para RSA.
PKI	Public Key Infrastructure — sistema de CAs y certificados.
RBAC	Role-Based Access Control — autorización por rol.
SAN	Subject Alternative Name — extensión X.509 con dominios/IPs adicionales.
TAG	Tag de autenticación de un AEAD — bytes que verifican integridad.
TLS 1.3	Transport Layer Security versión 1.3 (RFC 8446, 2018).
X.509	Estándar ITU-T para certificados de clave pública.

SecureRx — Documentación técnica completa

Generada con todo el contenido del proyecto, referencias a archivos y diagramas.